

Module : Data Mining

Filière : Cycle d'Ingénieur - 2A

RAPPORT DE TP 2

Manipulation et Visualisation

Bibliothèques Numpy, Pandas, Matplotlib et Seaborn

Réalisé par :

Youssef FELLAH

Encadré par :

Pr. Hamza ER-RAHMOUNY

Année universitaire : 2025/2026

1 Exercice 1 : Numpy

Dans cette première partie, nous initialisons l'environnement de calcul matriciel et répondons aux opérations basiques de création et de manipulation de tableaux (arrays) requises par l'énoncé.

```
# 1. Importer Numpy
import numpy as np

# 2. Créer un array de 10 zéros et de 10 uns
zeros_array = np.zeros(10)
ones_array = np.ones(10)

# 3. Créer un array des integers de 10 à 50, et un autre pour les entiers pairs
int_array = np.arange(10, 51)
even_int_array = np.arange(10, 51, 2)

# 4. Créer la matrice d'identité 3*3
identity_matrix = np.eye(3)

# 5. Créer une matrice 3*3 avec les nombres de 0 à 8
matrix_3x3 = np.arange(9).reshape(3, 3)

# 6. Générer un nombre au hasard entre 0 et 1
random_num = np.random.rand(1)

# 7. Générer un tableau de 25 nombres aléatoires (distribution normale standard)
normal_dist_array = np.random.randn(25)

# 8. Créer un tableau de 20 points linéairement espacés entre 0 et 1
linspace_array = np.linspace(0, 1, 20)

# 9. Créer la matrice mat 5*5 de nombres de 1 à 25 et calculer ses statistiques
mat = np.arange(1, 26).reshape(5, 5)

somme_mat = mat.sum() # 9.a
std_mat = mat.std() # 9.b
somme_colonnes = mat.sum(axis=0) # 9.c
```

Explications techniques :

- `np.arange(10, 51, 2)` : Génère un tableau d'entiers avec un pas de 2, permettant de ne conserver que les nombres pairs.
- `np.random.randn(25)` : Tire 25 échantillons d'une distribution normale centrée réduite (moyenne = 0, variance = 1).
- `mat.sum(axis=0)` : Effectue l'addition le long de l'axe des lignes (verticalement), ce qui produit la somme pour chaque colonne de la matrice.

2 Exercice 2 : Pandas

Cette section traite de la création structurée de DataFrames à l'aide de Pandas, ainsi que de l'extraction et du nettoyage de base.

```
import pandas as pd

# 1. Créer le DataFrame
np.random.seed(101)
mydata = np.random.randint(0, 101, (4, 3))
myindex = ['CANADA', 'USA', 'FRANCE', 'UK']
mycolumns = ['Jan', 'Feb', 'Mar']

df = pd.DataFrame(data=mydata, index=myindex, columns=mycolumns)

# 2. Exporter la DF sous format CSV
df.to_csv('mydata_export.csv')

# 3. Afficher les informations et les données statistiques
info_df = df.info()
stats_df = df.describe()

# 4. Supprimer la ligne 'FRANCE'
df_dropped = df.drop('FRANCE', axis=0)

# 5. Afficher uniquement les informations relatives à 'USA'
usa_data = df.loc['USA']

# 6. Donner la moyenne des valeurs du mois de janvier
jan_mean = df['Jan'].mean()
```

3 Exercice 3 : Pandas (Fichier Externe)

Manipulation avancée sur les valeurs manquantes à l'aide du fichier externe `movie_scores.csv`.

```
# 1. Créer le DataFrame
df_movies = pd.read_csv('movie_scores.csv')

# 2. Vérifier puis sélectionner les valeurs nulles
valeurs_nulles_totales = df_movies.isnull().sum()
lignes_avec_nulles = df_movies[df_movies.isnull().any(axis=1)]

# 3. Afficher les prénoms qui ne sont pas nuls
prenoms_valides = df_movies.loc[df_movies['first_name'].notnull(), 'first_name']

# 4. Trier les scores post-film DESC
df_sorted = df_movies.sort_values(by='post_movie_score', ascending=False)

# 5. Remplacer les valeurs nulles par 'Nouvelle valeur' dans un nouveau DF
df_filled = df_movies.fillna('Nouvelle valeur')

# 6. Supprimer toutes colonnes qui contiennent au moins une valeur nulle
df_clean_cols = df_movies.dropna(axis=1, how='any')
```

Explications techniques :

- `isnull().any(axis=1)` : Crée un filtre booléen permettant d'isoler uniquement les lignes possédant au moins une donnée manquante.
- `dropna(axis=1, how='any')` : Scanne les colonnes (`axis=1`) et supprime la colonne entière si elle contient ne serait-ce qu'une seule valeur NaN.

4 Exercice 4 : Matplotlib

Visualisation de séries temporelles avec comparaison de rendements obligataires sur différentes périodes.

```
import matplotlib.pyplot as plt

periode = ['1 Mois', '3 Mois', '6 Mois', '1 An', '2 An', '3 An', '5 An', '7 An', '10 An', '20 An', '30 An']
juillet16_2007 = [4.75, 4.98, 5.08, 5.01, 4.89, 4.89, 4.95, 4.99, 5.05, 5.21, 5.14]
juillet16_2020 = [0.12, 0.11, 0.13, 0.14, 0.16, 0.17, 0.28, 0.46, 0.62, 1.09, 1.31]

# 1. Tracer les deux courbes sur le même graphique avec légende
plt.figure(figsize=(10, 5))
plt.plot(periode, juillet16_2007, label='Juillet 2007', color='blue')
plt.plot(periode, juillet16_2020, label='Juillet 2020', color='orange')
plt.legend()
plt.title("Rendement par période")
plt.show()

# 2. Utiliser des subplots pour créer la figure
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))
```

```
axes[0].plot(periode, juillet16_2007, color='blue')
axes[0].set_title('Juillet 2007')
axes[1].plot(periode, juillet16_2020, color='orange')
axes[1].set_title('Juillet 2020')
plt.tight_layout()
plt.show()

# 3. Recréer le graphique qui utilise deux axes
fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.plot(periode, juillet16_2007, color='blue', label='Juillet 2007')
ax1.set_ylabel('Rendement 2007', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

ax2 = ax1.twinx()
ax2.plot(periode, juillet16_2020, color='red', label='Juillet 2020')
ax2.set_ylabel('Rendement 2020', color='red')
ax2.tick_params(axis='y', labelcolor='red')

plt.title("Comparaison avec deux axes Y")
plt.show()
```

5 Exercice 5 : Seaborn

Cette dernière section explore des relations statistiques multivariées avec la bibliothèque Seaborn, en exploitant le set de données `dm_office_sales.csv`.

```
import seaborn as sns

# Charger les données
df_sales = pd.read_csv('dm_office_sales.csv')

# 1. Afficher le graphe le Salary vs Sales
plt.figure(figsize=(8, 5))
sns.scatterplot(x='salary', y='sales', data=df_sales)
plt.title("Salary vs Sales")
plt.show()

# 2. Ajouter au graphe la division pour plus de précision (couleur)
plt.figure(figsize=(10, 6))
plot_with_division = sns.scatterplot(x='salary', y='sales', hue='division',
                                     data=df_sales)
plt.title("Salary vs Sales avec séparation par Division")
plt.show()

# 3. Exporter la figure sous format jpg
plot_fig = plot_with_division.get_figure()
plot_fig.savefig('salary_vs_sales_division.jpg', format='jpg', dpi=300)
```

Explications techniques :

- `sns.scatterplot(...)` : Mappe automatiquement la colonne "salary" sur l'axe X et "sales" sur l'axe Y pour générer un nuage de points.
- `hue='division'` : Paramètre de Seaborn qui analyse les catégories uniques présentes dans la colonne "division" et attribue automatiquement une couleur différente à chaque catégorie, tout en générant la légende.

Fin du rapport.